



Secure Software Design and Engineering (CY-321)

Security Policies

Security Policies

“The **security policy** of a system is a statement of the restrictions on access to objects and/or information transfer that a reference monitor is intended to enforce.”

More generally: “The **security policy** of a system is any formal statement of that system’s confidentiality, authenticity, and integrity (CIA) requirements.”

Example: The Unix password and group files are part of a Unix system’s security policy, because they determine

- when a user has successfully authenticated itself (A);
- which files the user can access (C)

Bell-LaPadula

Was designed in 1973 to codify existing military practices.

The system assigns a fixed numerical security level to each subject and object.

Needs a reference monitor that enforces two properties

When attached to an **object**, the label is called a **classification**.

Example: A file is labeled "Secret." This means: *this file contains information sensitive enough that only Secret-cleared (or higher) people should see it.*

When attached to a **subject**, the label is called a **clearance**.

Example: A user is granted "Secret" clearance. This means: *this person has been vetted/trusted to view information up to the Secret level.*

Bell-LaPadula

Used to maintain the confidentiality of security

Classification of subjects (users) and objects (files) are organized in a non-discretionary way with respect to different layers of secrecy

- Simple Confidentiality Rule
- Star Confidentiality Rule
- Strong Star Confidentiality Rule

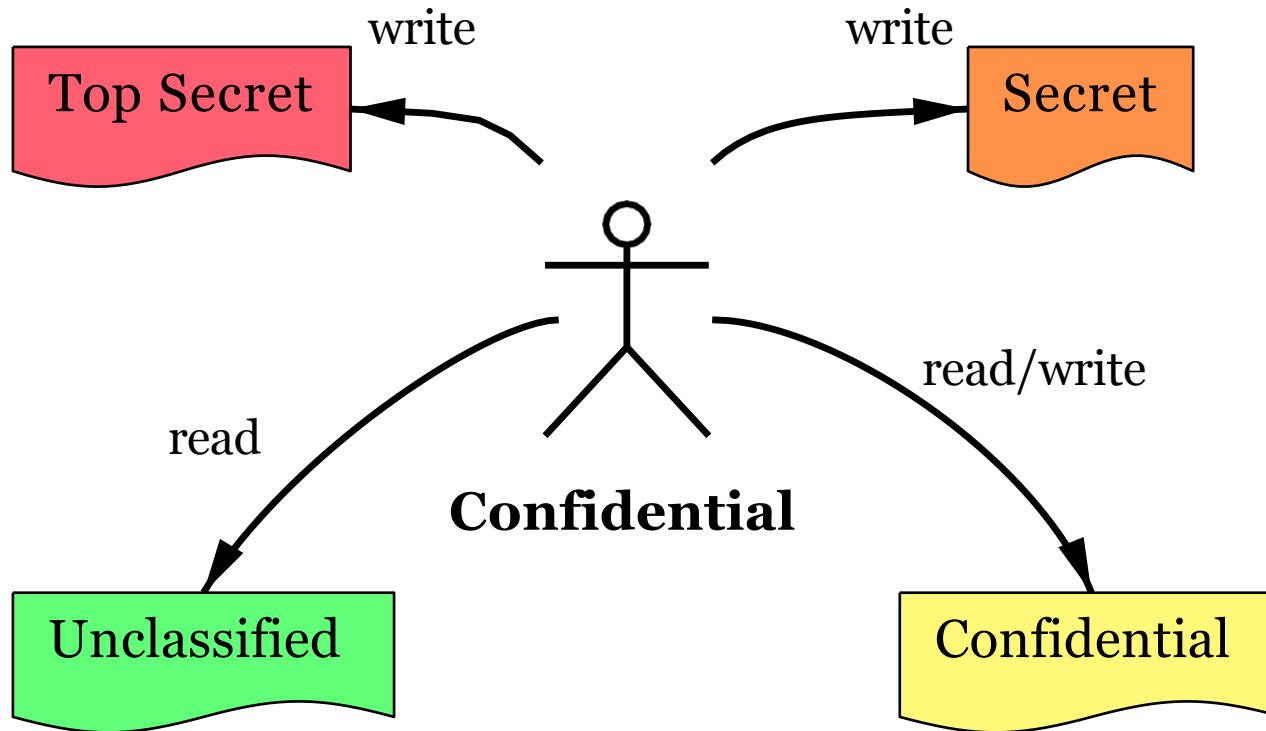
Bell-LaPadula

Written to address the “confinement problem”, i.e., the unwanted dissemination of information by Trojan Horse programs.

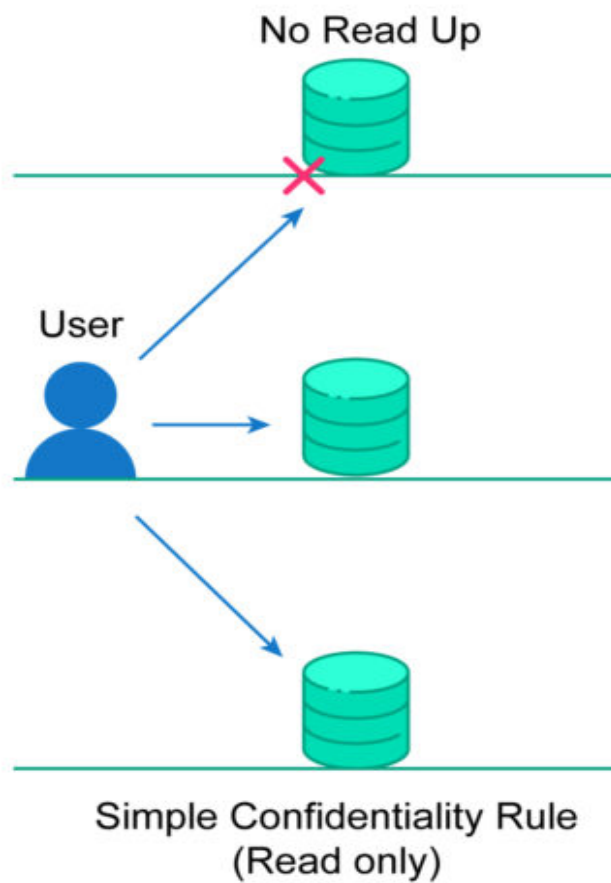
Trojan Horse program reads data on a multiuser mainframe and writes it to a location where an outsider can read it.

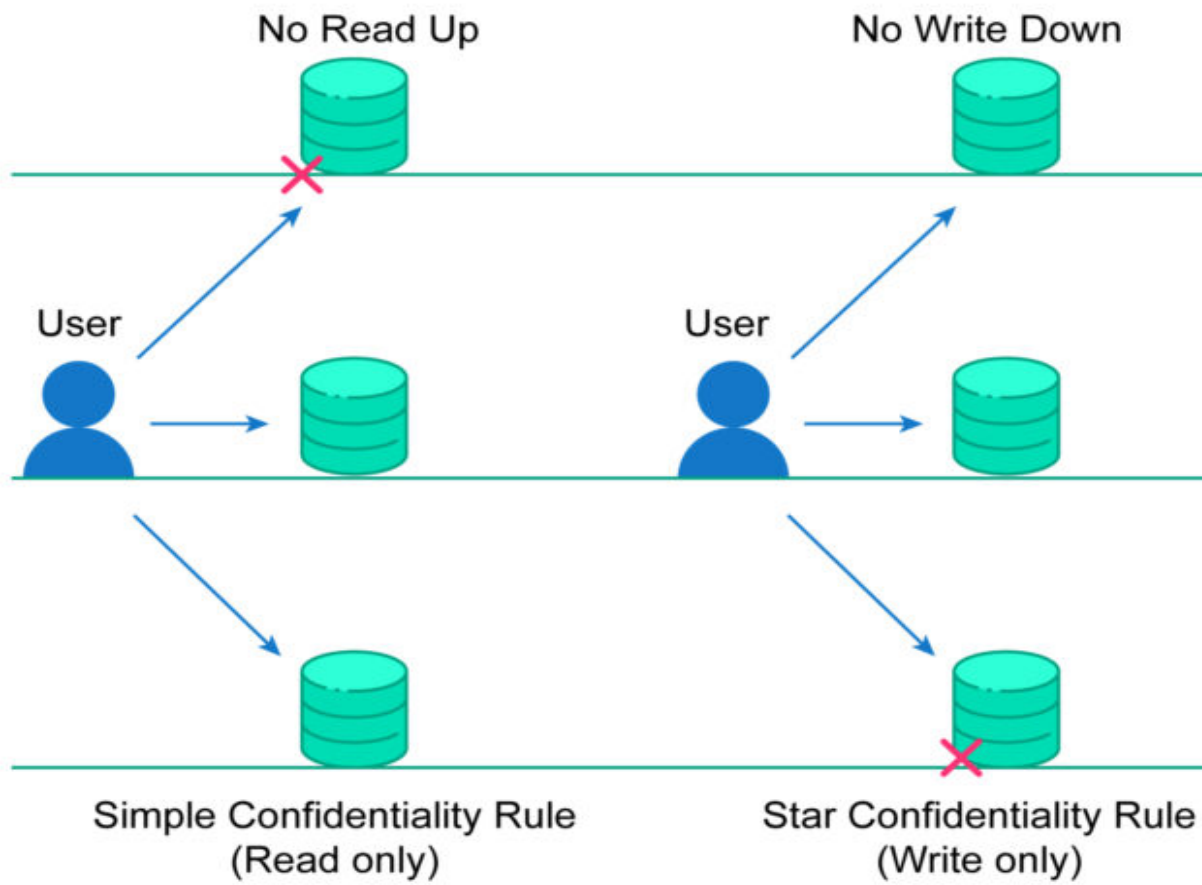
That's too oldfashioned? OK: a Word macro virus steals your sensitive data and uses Outlook express to send it over the Internet.

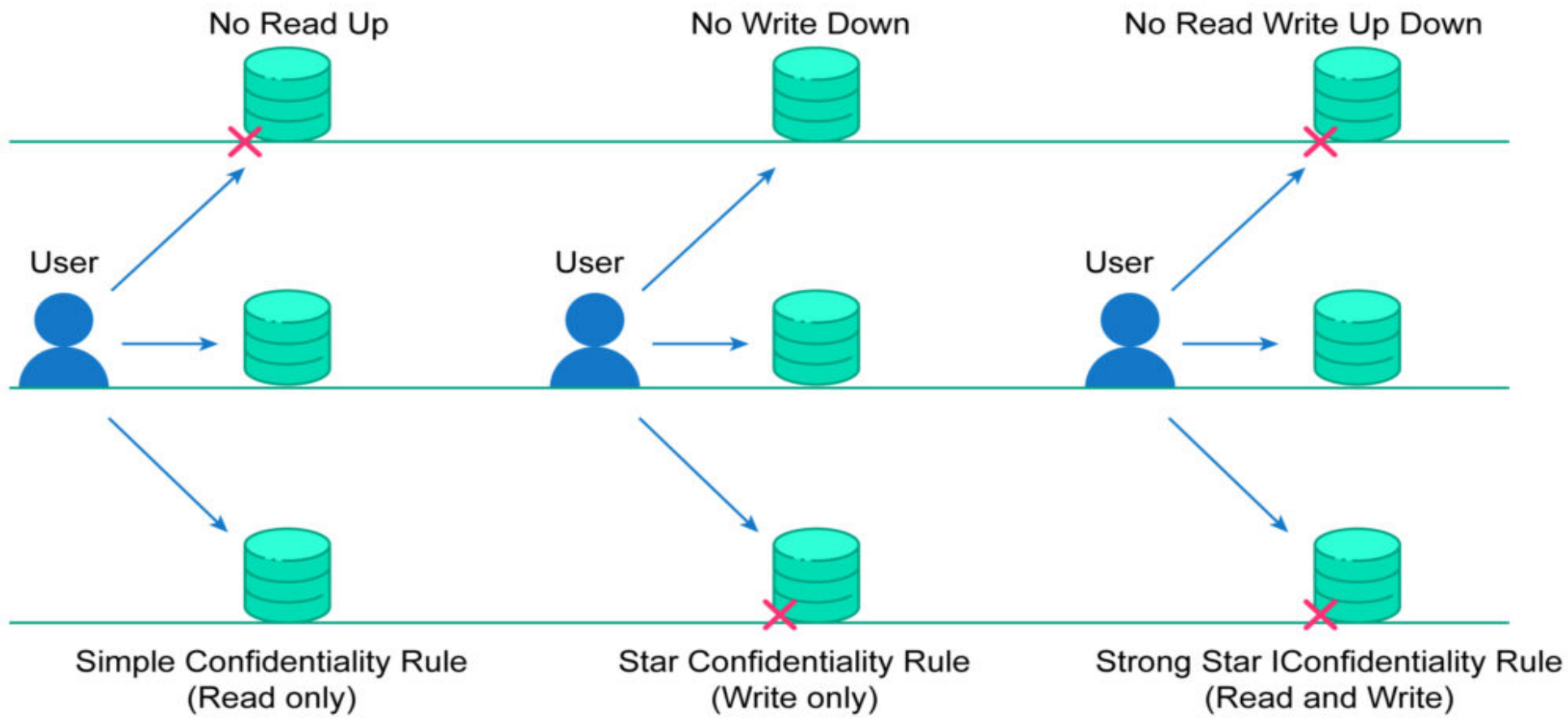
Example



- **The Simple Security Property**: a subject may read only objects that are at its own security level, or lower (“no read up”);
- **The *-Property (“Star Property”)**: a subject may write only objects that are at its own security level, or higher (“no write down”)







Practical Problem with Bell-LaPadula

Information tends to flow upward to the highest security level.

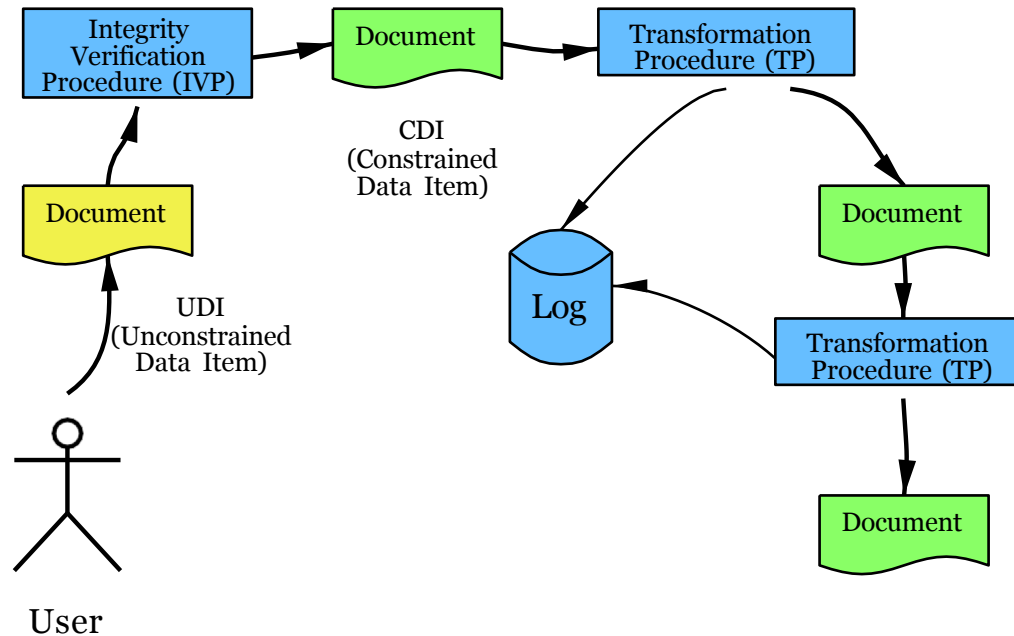
- Can't move down again because of ***-property**
- Once a document is accessed at level n , it can't be returned to level $m < n$.
- Users tend to have multiple copies at different levels.
- Also bad for integrity: Writes can't be verified.

Problems with Email and related services:

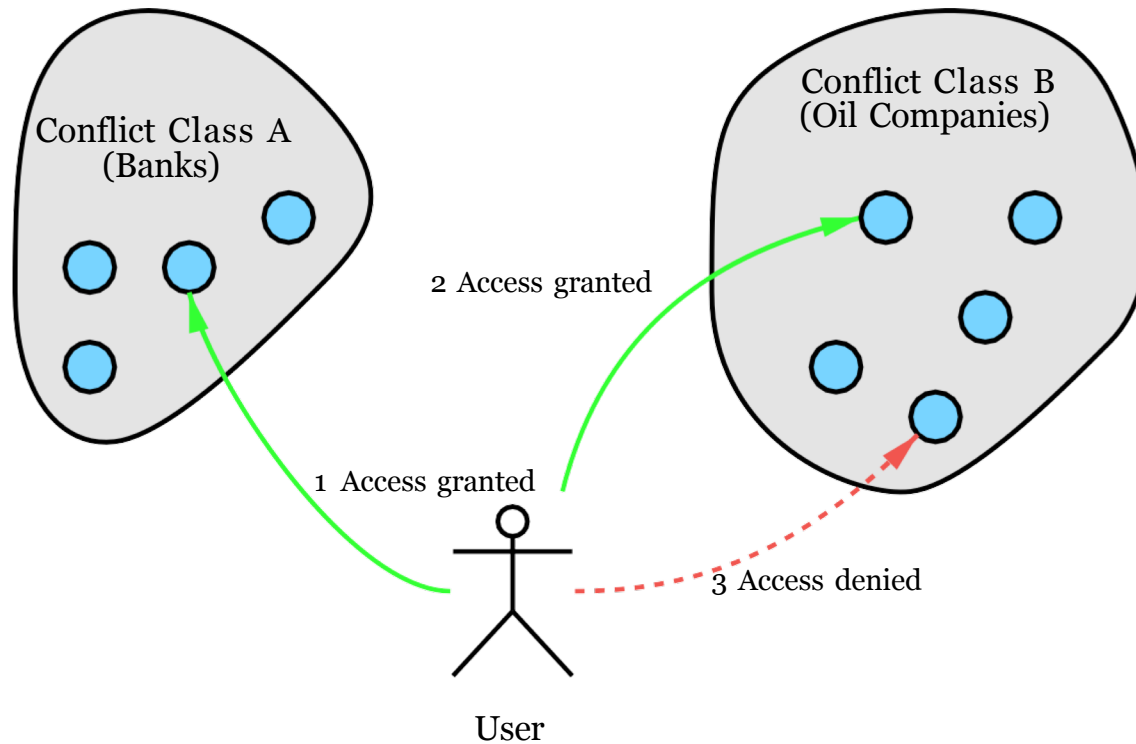
- A user level m isn't made aware of email messages at level $n > m$.
- A user at level n can see messages at level $m < n$, but can't reply to them

Clark-Wilson Model

How would a model look that is more concerned with *integrity* instead of confidentiality?



Chinese Wall Model



Chinese Wall Explained

Object groups are partitioned into distinct *conflict classes*.

“Partitioned” means that an object group belongs to exactly one conflict class.

A subject can get access to any object in any group, provided that it does not already have access to objects in another object group in the same conflict class.

Access to objects within an object group is unrestricted. Under these conditions, a conflict of interests cannot occur

Problems with Policy Models

It turns out that all these above policy models are equivalent to Bell–LaPadula.
(We won't prove this.)

“These basic models were intended to be used as general-purpose models and policies, applicable to all situations in which they were appropriate. Like other flexible objects such as rubber screwdrivers and foam rubber cricket bats, they give up some utility and practicality in exchange for their flexibility, and in practice tend to be extremely difficult to work with.” — Peter Gutmann

Solution: Apply policy models only to small and specific parts of the entire system. *Don't look for a silver bullet!*

REFERENCES

- Peter Gutmann, *Cryptographic Security Architecture*, Springer
- Peter Gutmann, *cryptlib Encryption Toolkit*,
<http://www.cs.auckland.ac.nz/~pgut001/cryptlib/>